Almanax

Security Scan Report

EveryFinance/smart-contracts-Stellar@080681fc2a8d84b9a92cfa609071dbca25b64e2b

Status: COMPLETE

Report Generated: 5/9/2026, 4:55:52 AM

Scan Date: May 8, 2026, 8:40 PM

Findings Summary

Severity	Count
Critical	**0**
High	**3**
Medium	**3**
Low	**6**
Info	**2**
Total	**14**

contracts/vault/src/lib.rs

HIGH Duplicate strategies inflate NAV and fees

Description:

`set_strategies` allows the same strategy address to appear multiple times. Since `nav()` sums `strategy_value_in_nav()` over the stored list, duplicates double-count a single position, inflating NAV/share price. Exploit path (manager-only, but concrete): manager sets the strategy list to include duplicates of an existing strategy with non-zero value → calls `deposit`/`withdraw` (even a tiny amount) to trigger `collect_fees()`, which mints management/performance fee shares based on inflated NAV → restores the correct (deduped) strategy list. The excess fee shares remain outstanding, diluting users and letting the manager redeem a larger fraction of real assets later.

Affected Code:

```
pub fn set_strategies(env: Env, caller: Address, strategies: Vec<Address>) {
    ...
    set_strategies(&env, &strategies);
}

fn nav(env: &Env, vault: &Address, base_asset: &Address) -> i128 {
    let strategies = get_strategies(env);
    let mut strategy_value: i128 = 0;
    for strategy in strate
... (truncated)
```

Recommendation:

> Enforce uniqueness in `set_strategies` (reject duplicates). Consider also enforcing a canonical ordering to prevent accidental duplication and simplify review.

MEDIUM Unbounded strategy list can DoS NAV

Description:

The strategy list length is unbounded. Every `deposit`, `withdraw`, `invest*`, `unwind*`, and `execute_trade` may call `nav()`, which iterates all strategies and performs cross-contract calls (`strategy.get_value`, and possibly `oracle.get_price`). A large list can exceed Soroban CPU/budget limits and make core user flows revert (liveness/DoS). Exploit path: a malicious/compromised manager sets an extremely large `strategies` vector → subsequent user `deposit/withdraw` calls run out of budget during NAV computation and fail, effectively freezing the vault.

Affected Code:

```
pub fn set_strategies(env: Env, caller: Address, strategies: Vec<Address>) {
    ...
    set_strategies(&env, &strategies);
}

fn nav(env: &Env, vault: &Address, base_asset: &Address) -> i128 {
    let strategies = get_strategies(env);
    for strategy in strategies.iter() {
        let v = Self::st
... (truncated)
```

Recommendation:

> Impose a reasonable maximum number of strategies (and optionally limit total external calls per operation). Consider caching NAV components or using a batched valuation model if many strategies are expected.

MEDIUM LP flag can brick NAV without checks

Description:

`set_lp_strategy` can mark any whitelisted strategy as LP without verifying it actually supports/has an internal oracle (`has_oracle`). When a strategy is flagged LP and later reports a non-zero value, `strategy_value_in_nav()` requires `strategy_has_oracle()` and will revert if false (or if the strategy doesn't implement the entrypoint), which can make `nav()` revert and consequently freeze deposits/withdrawals and other NAV-dependent operations. Exploit path: manager mistakenly (or maliciously) calls `set_lp_strategy(..., true)` on a non-LP strategy → once that strategy's `get_value(vault)` becomes non-zero, `nav()` reverts due to `LpStrategyOracleRequired`/failed `has_oracle` call → user deposit/withdraw becomes unusable.

Affected Code:

```
pub fn set_lp_strategy(env: Env, caller: Address, strategy: Address, is_lp: bool) {
    ...
    set_lp_strategy(&env, &strategy, is_lp);
}

fn strategy_value_in_nav(env: &Env, vault: &Address, strategy: &Address) -> i128 {
    let raw_v = strategy_get_value(env, strategy, vault);
    if is_lp_strate
... (truncated)
```

Recommendation:

> In `set_lp_strategy`, when `is_lp == true`, require `strategy_has_oracle(env, &strategy) == true` (and/or validate interface support). Consider making LP-flag changes a safer, two-step process to avoid accidental bricking.

contracts/mock_blend_pool/src/lib.rs

LOW Per-address persistent supply entries enable state bloat

Description:

The pool stores one persistent ledger entry per supplying `Address` (`DataKey::Supply(Address)`) and extends its TTL on each supply and even on `get_supply` reads. An attacker can create many funded addresses and call `submit` with small `REQUEST_SUPPLY` amounts to force unbounded growth of persistent state (and keep it alive via TTL bumps), increasing ledger/state footprint and potentially degrading operability/costs for integrators using this mock on shared networks.

Affected Code:

```
enum DataKey {
    Admin,
    Token,
    Supply(Address),
    TotalSupply,
}

for req in requests.iter() {
    ...
    if req.request_type == REQUEST_SUPPLY {
        ...
        env.storage().persistent().set(&supply_key, &(bal + req.amount));
        env.storage().persistent().extend_ttl(&supply_k
... (truncated)
```

Recommendation:

> If this contract can be deployed outside isolated tests, add anti-spam controls (minimum deposit, supplier cap, admin allowlist) and avoid extending TTL on read-only getters. Consider using instance storage (bounded set) or aggregating suppliers for mocks.

LOW Unchecked i128 math can wrap and corrupt accounting

Description:

Supply and total supply accounting uses unchecked `i128` addition/subtraction. In Soroban WASM release builds, Rust integer overflow wraps (two's complement) rather than reverting, which can silently corrupt stored balances (e.g., `bal + req.amount`, `total + req.amount`, `total - req.amount`). While reaching `i128` bounds is unlikely with typical token supplies, if a token uses very large amounts or this mock is reused in non-test contexts, wraparound can lock funds (negative balances causing perpetual `InsufficientSupply`) or desynchronize `TotalSupply` from actual pool balance.

Affected Code:

```
let bal: i128 = env.storage().persistent().get(&DataKey::Supply(from.clone())).unwrap_or(0);
let supply_key = DataKey::Supply(from.clone());
env.storage().persistent().set(&supply_key, &(bal + req.amount));
...
let total: i128 = env.storage().persistent().get(&DataKey::TotalSupply).unwrap_or(0);
env
... (truncated)
```

Recommendation:

> Use `checked_add/checked_sub` (or `soroban_sdk::panic_with_error!` on `None`) for all balance mutations to guarantee invariant-preserving behavior.

contracts/strategies/phoenix_lp/src/lib.rs

MEDIUM LP value falls back to raw share units

Description:

`get_value` returns the strategy's raw LP share-token balance (share units) when it cannot compute reserve-decomposition NAV (no oracle configured, `total_shares == 0`, or both pool reserves are zero). In these cases, the returned value is not denominated in the vault's base asset, so the vault's NAV/share-price can become badly mispriced. Concrete loss path (pool failure case): if a Phoenix pool is exploited or otherwise ends up with `reserve_a == 0 && reserve_b == 0` while LP shares remain outstanding, `get_value` will return a positive number (LP share units) instead of ~0 base value. The vault's NAV can be overstated, allowing withdrawals/mints/fee-accrual to occur at an inflated share price and potentially drain other liquid base-asset balances in the vault before the mismatch is discovered. Although the vault requires LP strategies to have an oracle configured before `invest_lp/unwind_lp`, this fallback still triggers in the "zero reserves" edge case even with an oracle set.

Affected Code:

```
pub fn get_value(env: Env, _vault: Address) -> i128 {
    let shares = token::Client::new(&env, &share_token).balance(&strategy);
    if shares == 0 {
        return 0;
    }

    let oracle = match get_oracle(&env) {
        Some(o) => o,
        None => return shares,
    };

    let (reserve_a, r
... (truncated)
```

Recommendation:

> Make `get_value` return `0` (or revert) when reserve-decomposition cannot be computed safely (e.g., `total_shares == 0` or reserves are both zero). If you need a debug/diagnostic view returning raw share units, expose it as a separate function so the vault/NAV path never consumes unit-ambiguous valu... (truncated)

LOW Internal share counter can desync from balance

Description:

The contract maintains an internal `total_shares` counter that is used to cap withdrawals (`share_amount > total_shares` reverts) and reported via `get_share_balance`, but `get_value` intentionally uses the live on-chain LP share-token balance instead of this counter. Because SEP-41 tokens are generally transferable, any third party can transfer Phoenix LP share tokens to this strategy contract address without calling `deposit_liquidity`. That increases `get_value` (vault NAV) but does not increase `total_shares`, meaning: - The vault/NAV path can overvalue the position (it counts donated/misdirected LP shares). - Those extra LP shares become effectively unrecoverable via this strategy because `withdraw` refuses to burn/redeem more than `total_shares`. This is a realistic “accounting drift + stuck funds” footgun (and can become a griefing vector if someone deliberately sends LP shares to inflate NAV while making that portion unwithdrawable through this strategy).

Affected Code:

```
pub fn withdraw(..., share_amount: i128, ..., from: Address, to: Address) -> (i128, i128) {
    let total = get_total_shares(&env);
    if share_amount > total {
        panic_with_error!(&env, PhoenixLpError::InsufficientShares);
    }
    ...
    let new_total = total.checked_sub(share_amount)...;
... (truncated)
```

Recommendation:

> Either (a) remove `total_shares` entirely and use the live LP share-token balance consistently for both withdrawal limits and reporting, or (b) add an admin-only `sync_total_shares_to_balance()` to reconcile `total_shares` to `balance(strategy)` and consider ignoring unsolicited LP-share transfers i... (truncated)

contracts/oracle/src/lib.rs

LOW Staleness check can be disabled to use old prices

Description:

The oracle allows `max_age_ledgers` to be set to `0`, which fully disables the staleness check in `load_fresh_price`. If the vault (or other consumers) relies on staleness to prevent using outdated prices, an oracle admin (or anyone who compromises the oracle admin key) can disable staleness and then allow old prices to remain usable indefinitely. This can enable stale-price exploitation during volatile markets (e.g., deposits/withdrawals/fee accrual computed off outdated NAV) or silently degrade safety assumptions that callers may expect from an “oracle”.

Affected Code:

```
pub fn set_max_age_ledgers(env: Env, max_age_ledgers: u32) {
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);
    let admin = get_admin(&env);
    admin.require_auth();
    set_max_age_ledgers(&env, max_age_ledgers);
}

fn load_fresh_price
... (truncated)
```

Recommendation:

> Do not allow `max_age_ledgers == 0` (enforce a minimum > 0), or gate disabling behind a stronger governance process (e.g., timelock / multisig) and emit an explicit on-chain signal/event for monitoring. If disabling is intentionally supported, consider returning an explicit flag in `get_max_age_ledg... (truncated)

contracts/vault/src/test.rs

INFO Insecure mock token hides authorization bugs

Description:

`MockToken` is used as a stand-in for SEP-41 in tests, but it omits critical authorization/allowance semantics: - `mint` has no admin check (any caller can mint arbitrary supply). - `approve` does not require `from.require_auth()`. - `transfer_from` ignores both spender auth and allowance/expiry checks. Because this file is `#![cfg(test)]`, this is not directly exploitable on-chain. However, it can produce false-positive test results and mask real production vulnerabilities (e.g., missing `require_auth`, incorrect allowance handling, or incorrect share-token admin assumptions) by allowing token movements/minting that would be rejected by a real SEP-41 implementation.

Affected Code:

```
pub fn mint(env: Env, to: Address, amount: i128) {
    let b: i128 = env
        .storage()
        .persistent()
        .get(&TKey::Balance(to.clone()))
        .unwrap_or(0);
    env.storage()
        .persistent()
        .set(&TKey::Balance(to.clone()), &(b + amount));
    let s: i128 = env

... (truncated)
```

Recommendation:

> Use the real `share_token`/SEP-41 implementation (or soroban token SDK) in integration tests, or harden `MockToken` to enforce: admin-only `mint/burn` (if applicable), `from.require_auth()` in `approve`, and allowance + spender auth + expiry enforcement in `transfer_from`. This makes tests meaningfu... (truncated)

contracts/factory/src/lib.rs

LOW Admin can be transferred without acceptance

Description:

`set_admin` lets the current admin unilaterally set `new_admin` without requiring `new_admin.require_auth()`. If `new_admin` is an address that cannot/will not authorize (e.g., an uninitialized contract address, wrong key, or a third party that never accepts), the Factory becomes permanently admin-locked: no further vault registrations/removals or admin rotation can occur. Exploit path: convince/phish the current admin into calling `set_admin(..., new_admin)` to an unusable address (or a typo). After the transaction, all future admin-gated functions become impossible to call, effectively freezing registry management.

Affected Code:

```
pub fn set_admin(env: Env, caller: Address, new_admin: Address) {
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);

    caller.require_auth();
    if caller != get_admin(&env) {
        panic_with_error!(&env, FactoryError::NotAdmin);

... (truncated)
```

Recommendation:

> Require acceptance by adding `new_admin.require_auth()` (two-step transfer: `set_pending_admin` + `accept_admin` is even safer), and consider emitting both old/new admin in the event.

contracts/factory/src/storage.rs

LOW Expired persistent keys can corrupt registry state

Description:

The registry relies on persistent entries (`VaultByIndex`, `VaultPosition`, `IsRegistered`) staying alive, but their TTL is only extended when each specific key is accessed. If some entries expire (e.g., long-term lack of pagination calls reaching older indices), `VaultCount` in instance storage can still indicate those vaults exist while the underlying persistent keys have been deleted by TTL expiry. Impact paths: - `get_vaults()` iterates `i < total` and calls `get_vault_by_index()`, which calls `extend_ttl` and then `unwrap()` on the persistent key. If the entry expired, this will trap, causing a DoS for enumeration. - `register_vault()` uses `get_is_registered()` as the duplicate guard. If `IsRegistered(vault)` expired, the same vault can be re-registered again, creating duplicates and inconsistent reverse indices. While this is primarily a liveness/state-integrity issue (not direct fund loss), it can break off-chain discovery, impede admin operations, and cause registry inconsistency over time.

Affected Code:

```
fn bump_persistent(env: &Env, key: &DataKey) {
    env.storage()
        .persistent()
        .extend_ttl(key, PERSISTENT_LIFETIME_THRESHOLD, PERSISTENT_BUMP_AMOUNT);
}

pub fn get_vault_by_index(env: &Env, idx: u32) -> Address {
    let key = DataKey::VaultByIndex(idx);
    bump_persistent(env, &k
... (truncated)
```

Recommendation:

> Make registry resilient to TTL expiry: (a) avoid `extend_ttl`/`unwrap()` on missing keys by checking existence and returning a controlled error, and (b) add a maintenance/bump function to extend TTL for a caller-specified index range (or require index-range access as part of regular ops) so old entr... (truncated)

contracts/oracle/src/storage.rs

HIGH Config entries can expire and brick oracle

Description:

The oracle stores critical configuration (`Admin`, `MaxAgeLedgers`) in **persistent** storage, but their TTL is only extended on `set_admin` / `set_max_age_ledgers` writes. Reads (`get_admin`, `get_max_age_ledgers`) do not extend TTL. As a result, after `PERSISTENT_BUMP_AMOUNT` ledgers (~30–36 days) without calling `set_admin` / `set_max_age_ledgers`, these entries can expire even while the contract is still being used. Impact: - Once `MaxAgeLedgers` expires, `get_price()` will revert (via `NotInitialized`), causing downstream vault NAV/deposit/withdraw flows that depend on oracle prices to DoS. - Once `Admin` expires, admin-gated operations (e.g., `set_price`) become impossible because the contract can no longer load an admin address to authenticate, effectively permanently bricking price updates. Exploit path: no special attacker capabilities are required; simply waiting for TTL expiry is sufficient. A third party can keep calling `get_price` (which bumps only per-asset `Price` keys) to keep price entries alive while `Admin`/`MaxAgeLedgers` silently expire, making the failure more surprising and harder to diagnose.

Affected Code:

```
pub fn get_admin(env: &Env) -> Address {
    env.storage()
        .persistent()
        .get(&DataKey::Admin)
        .unwrap_or_else(|| panic_with_error!(env, OracleError::NotInitialized))
}

pub fn get_max_age_ledgers(env: &Env) -> u32 {
    env.storage()
        .persistent()
        .get(&DataK
... (truncated)
```

Recommendation:

> Extend TTL for `Admin` and `MaxAgeLedgers` on reads (similar to `get_price_data`), or move these config fields to instance storage and rely on the contract's existing instance TTL bump on every entrypoint call. Also consider bumping these config keys' TTL from every public method to prevent accident... (truncated)

contracts/integration_tests/src/test_vault_lifecycle.rs

INFO Auth mocking hides missing require_auth regressions

Description:

The test harness globally calls `env.mock_all_auths()`, which bypasses all `require_auth` checks for the duration of these tests. As a result, these integration tests will still pass even if a production change accidentally removes or incorrectly scopes `require_auth` on critical entrypoints like `deposit`/`withdraw`/privileged manager actions, reducing the chance of catching authorization regressions before deployment. Exploitation path: a future code change unintentionally weakens/removes `require_auth` in the Vault/ShareToken contracts; CI still passes because these integration tests bypass auth; the weakened contract gets deployed; an attacker can then call the affected entrypoint without the intended signature/authorization (e.g., withdrawing from another `from` address if that check regresses).

Affected Code:

```
let env = Env::default();
env.mock_all_auths();
```

Recommendation:

Add at least one integration test suite (or specific tests in this file) that does NOT use `mock_all_auths`, and instead sets explicit auth via Soroban testutils (`env.set_auths(...)` / recorded auth) to assert that unauthorized callers fail for each privileged path and that `deposit/withdraw` enfor... (truncated)

contracts/strategies/soroswap_lp/src/lib.rs

HIGH Separate strategy manager can manipulate vault NAV

Description:

The strategy introduces an additional privileged role (`manager`) that is independent of the vault's on-chain manager. This `manager` can (1) set/replace the oracle used by `get_value()` (affecting vault NAV/share price), and (2) pause/unpause the strategy (blocking vault unwinds/withdrawals). Because `manager` is set once at initialization and is not required to equal the vault manager (and cannot be rotated by the vault), compromise of this key is sufficient to: - **Manipulate NAV**: call `set_oracle()` to point to a malicious oracle that returns inflated prices for `asset_a`/`asset_b`, increasing `get_value()` and vault NAV. An attacker holding vault shares can then withdraw against inflated NAV, draining base assets from the vault. - **Cause DoS / lock funds**: call `pause()` while the strategy holds LP; vault cannot unwind liquidity for user withdrawals, and the vault may be unable to remove the strategy if it enforces "cannot remove when position active". This expands the trusted surface area beyond the vault Manager/Trader roles described in the threat model, and creates a single-key failure mode for both valuation and liveness.

Affected Code:

```
pub fn initialize(
    env: Env,
    vault: Address,
    asset_a: Address,
    asset_b: Address,
    lp_token: Address,
    router: Address,
    manager: Address,
    name: String,
) {
    // ...
    let vault_manager: Address = env.invoke_contract(
        &vault,
        &Symbol::new(&env, "get_ma
... (truncated)
```

Recommendation:

> Bind privileged controls to the vault's governance rather than a separate immutable strategy manager.
> Options:
> - Require the **vault address** (or vault manager fetched from the vault at call time) to authorize `set_oracle/pause/unpause`.
> - Add a `set_manager(new_manager)` function callable only by ... (truncated)